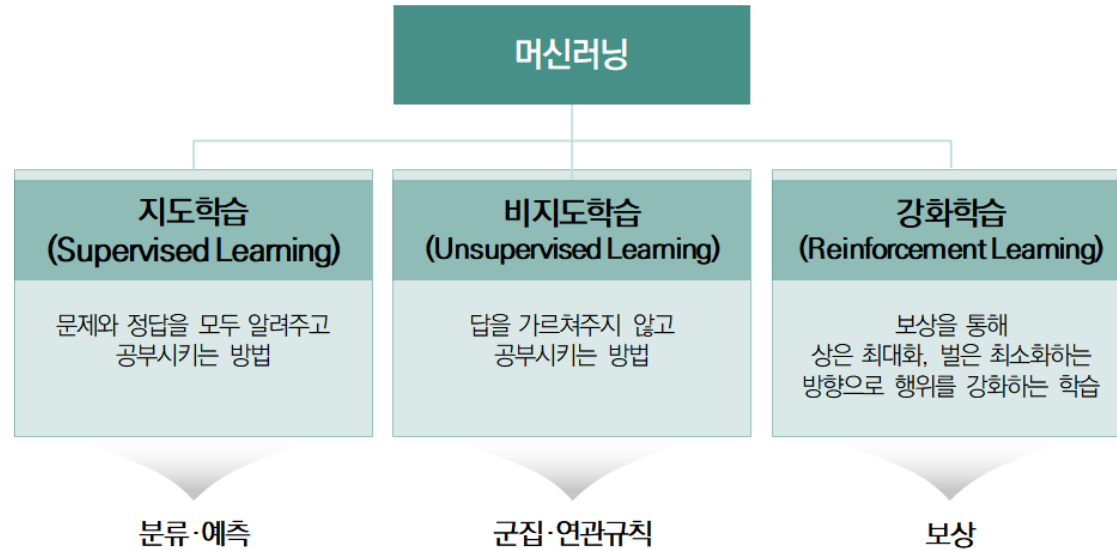


체스 Agent 제작 2주차

트리 탐색 기반 에이전트 구현

1. 강화학습이란?

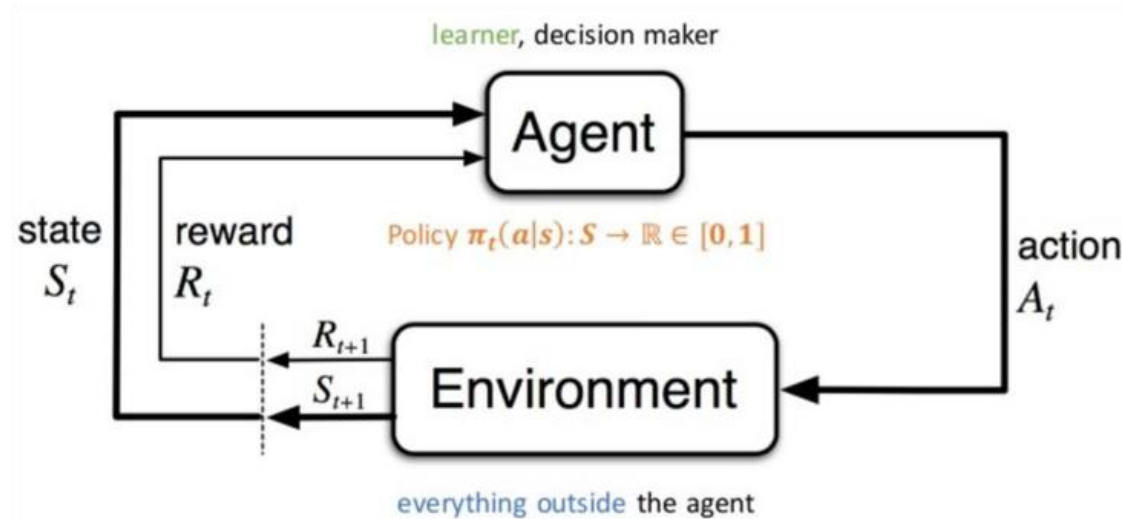
- 인공지능이 별도의 데이터 없이 환경과 상호작용하며 시행착오를 통해 학습하는 과정
- Q-learning, Actor critic method 등의 알고리즘 존재



<그림 1>

1. 강화학습이란?

- 강화학습의 학습 요소
 - Agent: 환경과 상호작용하며 의사결정 내리는 주체, Player
 - State: 특정 시점에 Agent가 관찰한 환경 상태
 - Action: 에이전트가 판단하여 취할 수 있는 선택지
 - Reward: 에이전트가 내린 판단에 따른 보상값, 게임 Agent에서는 주로 신경망 사용
 - Policy: 에이전트가 주어진 정보를 활용하여 Action을 선택하는 전략



<그림 2>

2. Q-learning

- 이번 Agent에 적용할 알고리즘: Q-learning
- 행동-가치 함수: Q-learning의 핵심 요소: 행동-가치 함수 $Q(s,a)$
- $Q(s,a) \rightarrow$ 상태 s 에서 a 라는 행동을 했을 때 얼마나 좋은 결과가 나오는지 숫자로 표현하는 함수
- Q-learning은 시행(episode)를 반복하며 $Q(s,a)$ 값들을 채워 가면서 기대 보상표를 채워 나가는 과정
- Q-learning 업데이트 식: <그림 3>의 식

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

The diagram illustrates the Q-learning update equation with color-coded components and annotations:

- New Q-value estimation** (green line under $Q(S_t, A_t)$ on the left)
- Former Q-value estimation** (blue line under $Q(S_t, A_t)$ on the right)
- Learning Rate** (red line under α)
- Immediate Reward** (orange line under R_{t+1})
- Discounted Estimate optimal Q-value of next state** (purple line under $\gamma \max_a Q(S_{t+1}, a)$)

Below the equation, two horizontal bars represent:

- TD Target** (blue bar, spanning from the former Q-value estimation to the discounted estimate)
- TD Error** (yellow bar, spanning from the former Q-value estimation to the end of the equation)

<그림 3>

2. Q-learning

- Q-learning 업데이트 식(Bellman 식)

- $Q(S_t, A_t)$: 게임 중 S 상태에서 t번째 수로 A를 두었을 때 기대할 수 있는 결과

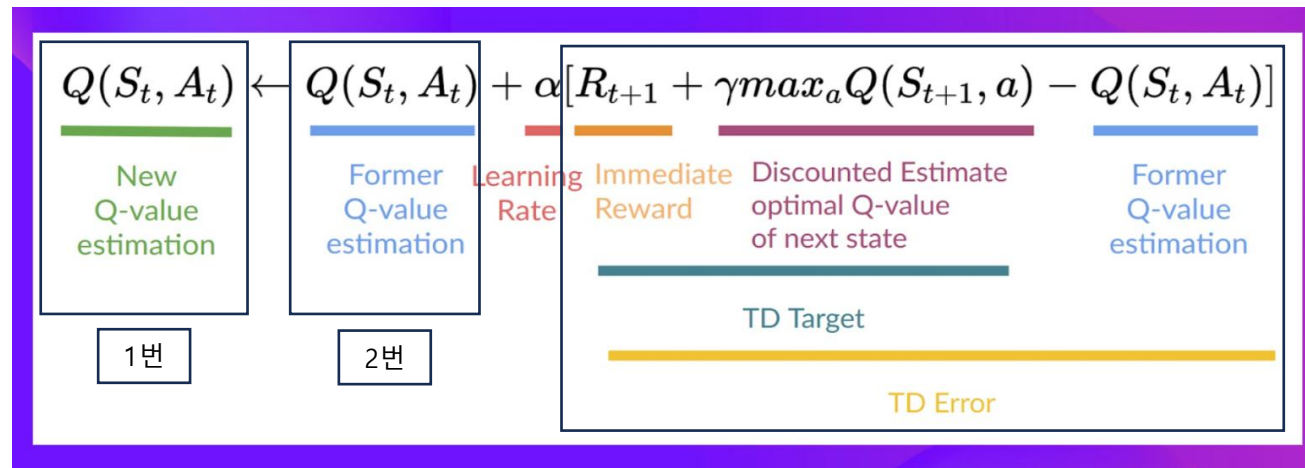
1번 $Q(S_t, A_t)$: 탐색을 통해 새롭게 변경되는 값, 2번 $Q(S_t, A_t)$: 기존에 있었던 $Q(S_t, A_t)$ 값

- TD error(Temporal-Difference error): 탐색을 통해 새롭게 알게 된 보상값(TD target)과 기존의 $Q(S_t, A_t)$ 와의 차이

- TD target의 구성요소

1) R_{t+1} : A_t 를 통해 얻는 즉각적인 보상(ex. A_t 를 통해 파낸 상대 기물의 점수), 2) $\max_a Q(S_{t+1}, a)$: S_{t+1} 에서 취할 수 있는 모든 행동 a 중 Q함수의 최댓값, 3) γ : 할인율(다음 수의 Q값 가치를 현재 수에 얼마 정도로 반영할지에 대한 수치), 4) α : 학습률

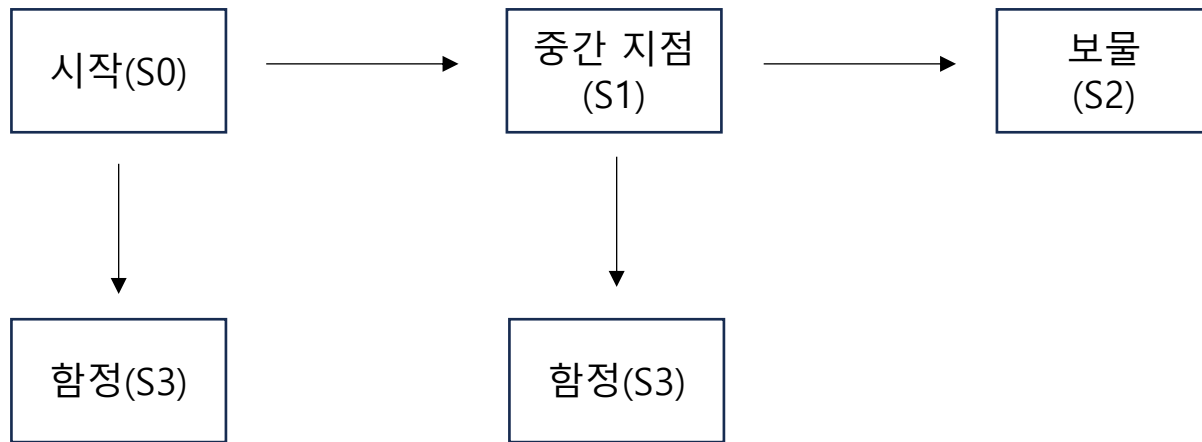
참고) t 는 상태와 행동 자체의 일부가 아니라 시점을 나타내는 첨자이다. 따라서 현재 보드 상태와 차례가 동일한 상태 S에서 동일한 행동 A를 선택했다면, 몇 번째 수인지와 관계없이 같은 $Q(S, A)$ 값을 업데이트할 수 있다.



<그림 3>

2. Q-learning

- Q-learning 예시
- 예시: 보물찾기 게임
- 규칙: <그림 4>의 지형 / 함정 또는 보물에 도달하면 게임 종료 / 함정 보상: -10, 보물 보상: +10, 일반 이동 보상: 0
- 가능한 이동은 오른쪽/아래쪽 이동
- <표 1>: 초기 $Q(s, a)$ 표 / 위치는 S , 이동(행동)은 a 에 해당함



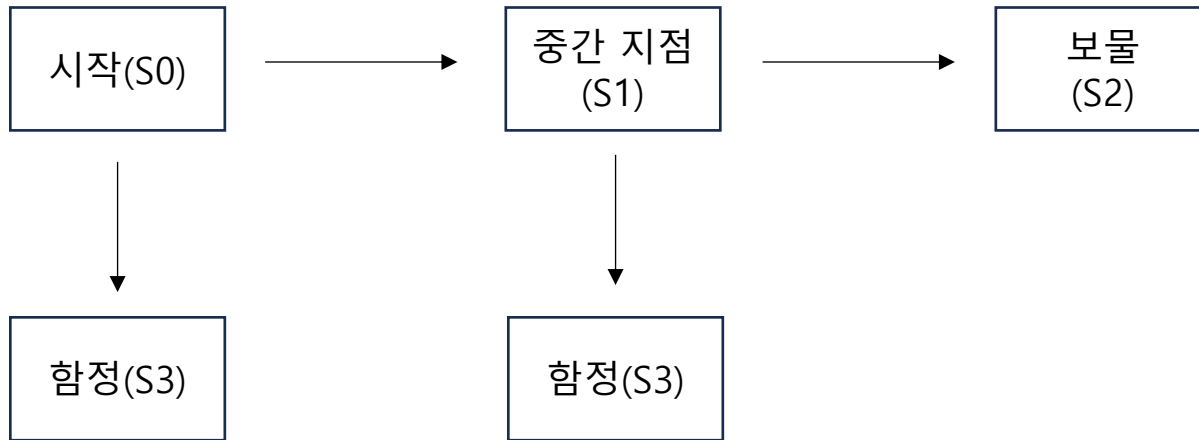
<그림 4>

위치(=상태, S)	오른쪽	아래쪽
시작(S0)	0	0
중간 지점(S1)	0	0

<표 1>: 초기 $Q(s, a)$ 표

2. Q-learning

- Q-learning 예시
- $Q(s,a)$ 업데이트 과정: 학습률(α) = 0.5, 할인률(γ) = 0.9
- 1번째 episode - 1단계: S0에서 오른쪽으로 이동 후 Q값 업데이트
 - $Q(S0, \text{오른쪽})_{\text{새값}} = Q(S0, \text{오른쪽})_{\text{이전값}} + 0.5(R + 0.9 * (\max Q(S1, \text{오른쪽}/\text{아래쪽})) - Q(S0, \text{오른쪽})_{\text{이전값}}) = 0$ (참고: 즉시 얻는 보상이 0이므로 R값도 0임)
- 1번째 episode 2단계: S1에서 오른쪽으로 이동 후 Q값 업데이트: 같은 방식을 거치면 $Q(S1, \text{오른쪽}) = 5$ 로 변경(즉시 얻는 보상이 10이므로 $R = 10$ 에 의한 결과)
- 1, 2단계를 통해 에이전트가 S2에 도착하였으므로 1번째 episode 종료



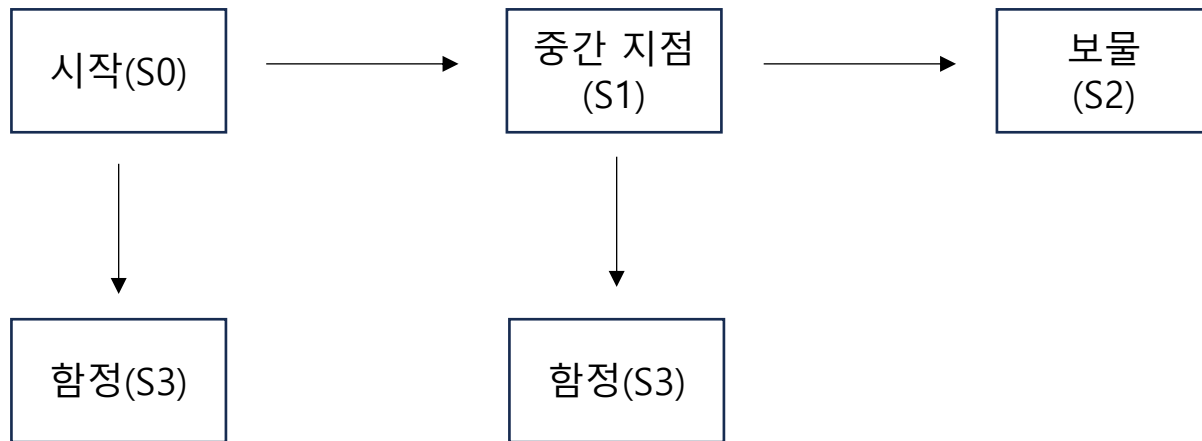
<그림 4>

위치(=상태, S)	오른쪽	아래쪽
시작(S0)	0	0
중간 지점(S1)	5	0

<표 2>: 첫 episode 후 $Q(s,a)$ 표

2. Q-learning

- Q-learning 예시
- $Q(s,a)$ 업데이트 과정: 학습률(α) = 0.5, 할인률(γ) = 0.9
- 새로운 episode 시작, 다시 S0에서 시작
- 2번째 episode - 1단계: S0에서 오른쪽으로 이동
 - $Q(S0, \text{오른쪽})_{\text{(새값)}} = Q(S0, \text{오른쪽})_{\text{(이전값)}} + 0.5(R + 0.9 * (\max Q(S1, \text{오른쪽/아래쪽})) - Q(S0, \text{오른쪽})_{\text{(이전값)}}) = 0 + 0.5 * (0 + 0.9 * 5 - 0) = 2.25$
- 2번째 episode - 2단계: S1에서 오른쪽으로 이동 $\rightarrow Q(S1, \text{오른쪽}) = 7.5$ 로 변경



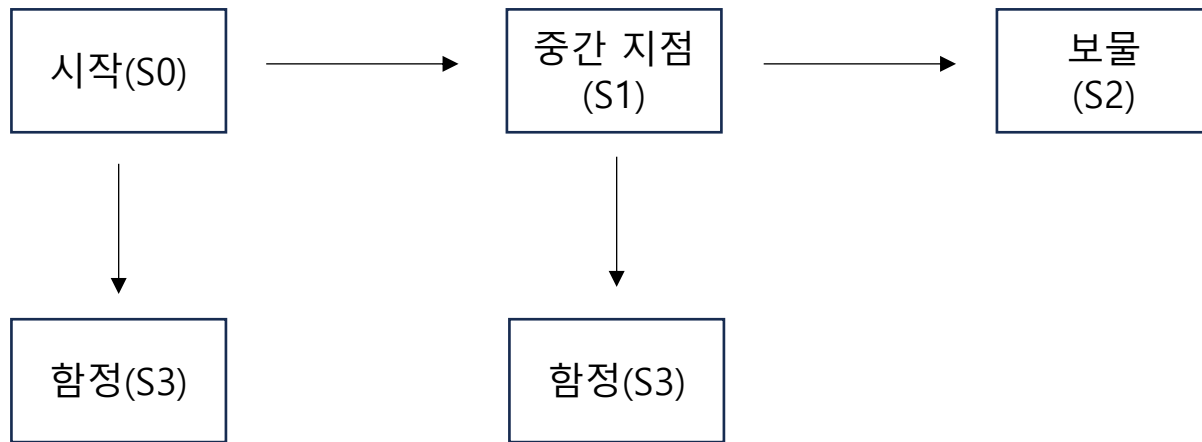
<그림 4>

위치(=상태, S)	오른쪽	아래쪽
시작(S0)	2.25	0
중간 지점(S1)	7.5	0

<표 3>: 2번째 episode 후 $Q(s,a)$ 표

2. Q-learning

- Q-learning 예시
- $Q(s,a)$ 업데이트 과정: 학습률(α) = 0.5, 할인률(γ) = 0.9
- 새로운 episode 시작, 다시 S0에서 시작 - 이번에는 함정 탐색
- 3번째 episode - 1단계: S0에서 오른쪽으로 이동
 - $Q(S0, \text{오른쪽})_{\text{(새값)}} = Q(S0, \text{오른쪽})_{\text{(이전값)}} + 0.5(R + 0.9 * (\max Q(S1, \text{오른쪽/아래쪽})) - Q(S0, \text{오른쪽})_{\text{(이전값)}}) = 2.25 + 0.5 * (0 + 0.9 * 7.5 - 2.25) = 4.5$
- 3번째 episode - 2단계: S1에서 아래쪽으로 이동 -> $Q(S1, \text{아래쪽}) = -5$ 로 변경



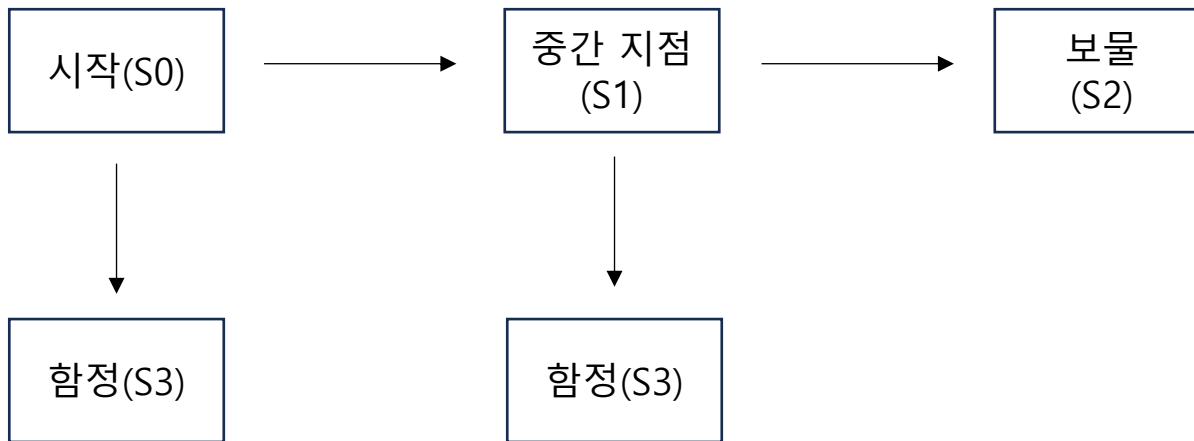
<그림 4>

위치(=상태, S)	오른쪽	아래쪽
시작(S0)	4.5	0
중간 지점(S1)	7.5	-5

<표 4>: 3번째 episode 후 $Q(s,a)$ 표

2. Q-learning

- Q-learning 예시
- $Q(s,a)$ 업데이트 과정: 학습률(α) = 0.5, 할인률(γ) = 0.9
- 무한한 Episode를 거친 후의 $Q(s,a)$ 표는 <표 5>와 같다.



<그림 4>

위치(=상태, S)	오른쪽	아래쪽
시작(S0)	9	-10
중간 지점(S1)	10	-10

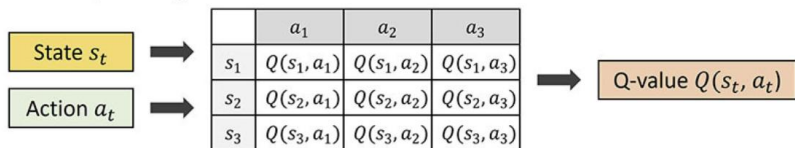
<표 5>: 최종 $Q(s,a)$ 표

3. DQN(Deep Q-learning)

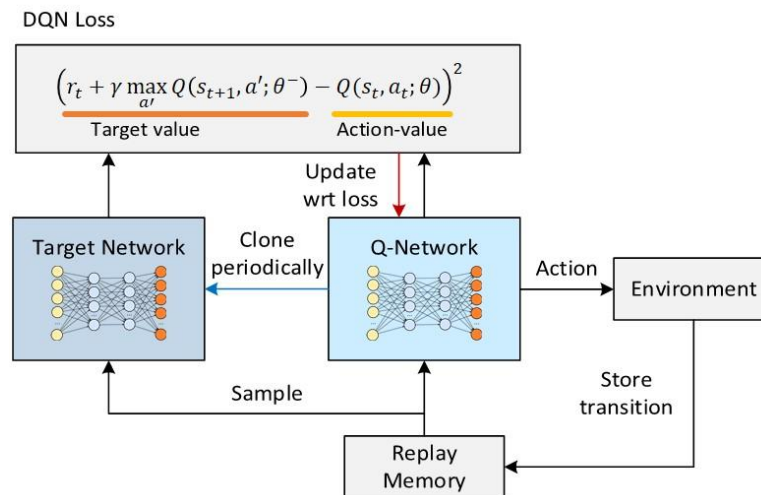
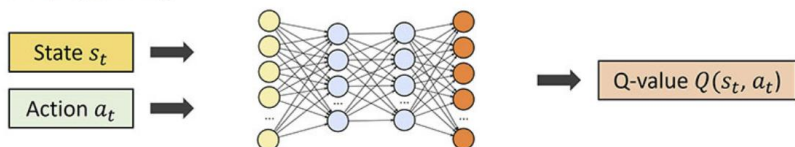
- DQN(Deep Q-learning)

- 체스와 같은 게임에서 모든 (S,A)에 대한 Q table을 만드는 것은 불가능함
- $Q(s, a)$ 를 신경망을 활용하여 근사함
- Q-learning의 Bellman 식으로 목표 Q값을 계산하고, TD error를 활용해 손실 함수를 정의
- 경사하강법, 역전파를 활용해 신경망 가중치 업데이트

Classic Q-learning



Deep Q-learning



3. DQN(Deep Q-learning)

• DQN의 학습 과정

- 1단계: t번째 수에서 현재 체스판 상태 S_t 와 가능한 행동들 A_t 의 쌍을 신경망에 입력
- 2단계: 신경망이 각각의 (S_t, A_t) 쌍에 대한 Q값을 출력
- 3단계: ϵ -greedy 방식으로 수를 선택하여 수를 선택함($1 - \epsilon$ 의 확률로 $Q(S_t, A_t)$ 가 최대가 되게 하는 A_t 선택, 추가 학습을 위해 ϵ 의 확률로 무작위 A_t 선택)
- 4단계: A_t 를 두었을 때 새롭게 R_{t+1} 과 상태 S_{t+1} 이 주어짐
- 5단계: TD target(y_t)값 계산 후 손실값(L, TD error 제곱) 계산(그림 6)
- 6단계: 신경망 값 업데이트(그림 7)

$$y_t = R_{t+1} + \gamma \max_{a'} Q_{\text{target}}(S_{t+1}, a')$$

$$L = [y_t - Q_{\text{online}}(S_t, A_t)]^2$$

<그림 6>

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L$$

<그림 7>

3. DQN(Deep Q-learning)

- <그림 6>, <그림 7> 부연설명

- <그림 6>: Q_{online} : 현재 학습 중인 신경망이 예측한 Q값, Q_{target} : 목표 Q값을 계산할 때 사용하는 별도의 신경망이 출력한 값

참고) Q_{online} 과 Q_{target} 을 분리하는 이유: 학습 중 목표값이 움직이는 현상을 방지하기 위함, Q_{online} 이 업데이트되고 나면 일정 주기 이후 새로운 Q_{target} 으로 사용됨

- <그림 7>: θ : 신경망의 가중치, α : 학습률, $\nabla_{\theta} L$: 손실 함수 L 을 신경망의 모든 가중치 θ 에 대해 편미분한 값들을 모은 벡터(예시는 3개의 θ 를 가진 신경망을 활용함)
- $\nabla_{\theta} L$ 의 의미: 손실이 가장 빠르게 증가하는 방향

Ex) 간단한 신경망 $y' = \theta_1 x + \theta_2$ 가 있고, 초기 $\theta_1=1, \theta_2=0$ 이라 하자($\theta = [1,0]$). 이때 $x=1$ 일때 예측값 $y'=1$ 이고, 실제 y 값은 3으로 관측되었다고 하자. 손실함수는 $L = (y - (\theta_1 x + \theta_2))^2 = (y - y')^2$ 이고, $\frac{dL}{d\theta_1} = -2x(y - y')$, $\frac{dL}{d\theta_2} = -2(y - y')$ 이다. 현재 상황에서의 x, y, y' 값을 대입하면 $\nabla_{\theta} L = [-4, -4]$ 이다. 손실을 빠르게 줄이기 위해서는 $\nabla_{\theta} L$ 의 반대 방향으로 가중치를 업데이트해야 한다. 학습률 $\alpha = 0.1$ 이라 하면 새롭게 계산된 가중치는 $\theta = [1.4, 0.4]$ 로 변화한다.

- 심층 신경망의 가중치 업데이트 과정은 경사하강법 관련해서 별도로 찾아보시길 권장합니다.

$$y_t = R_{t+1} + \gamma \max_{a'} Q_{\text{target}}(S_{t+1}, a')$$

$$L = [y_t - Q_{\text{online}}(S_t, A_t)]^2$$

<그림 6>

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L$$

$$\nabla_{\theta} L = \begin{bmatrix} \frac{\partial L}{\partial \theta_1} \\ \frac{\partial L}{\partial \theta_2} \\ \frac{\partial L}{\partial \theta_3} \end{bmatrix}$$

<그림 7>

4. DQN Agent 코딩 준비

- DQN Agent의 구성요소
- 새롭게 추가된 파일: chessboard_for_dqn.py, dqn_agent.py, dqn_encoder.py, replay_buffer.py, dqn_model.py, train_dqn.py
- chessboard_for_dqn: 원활한 훈련을 위해 무승부 규칙을 추가(gamestate 업데이트 사항, dqn_agent 외의 다른 agent와도 호환됨)
- dqn_agent.py: Q-learning 알고리즘에 의해 수를 선택하는 Agent 파일
- dqn_encoder.py: 신경망이 게임 상태와 착수 행위를 해석할 수 있게끔 tensor 형태로 변환해주는 파일
- replay_buffer.py: 생성된 훈련 데이터를 가공하는 파일
- train_dqn.py: Agent 간의 대국으로 훈련 데이터를 생성하는 파일

```
▼ black_white
  > __pycache__
  • __init__.py
  • chessboard_for_dqn.py
  • chessboard_for_tree.py
  • chessboard.py
  • player.py
```

```
▼ agent_file
  > __pycache__
  • __init__.py
  • alphabeta_agent.py
  • base.py
  • dqn_agent.py
  • random_agent.py
```

```
▼ encoder
  > __pycache__
  • __init__.py
  • dqn_encoder.py
▼ memory
  > __pycache__
  • __init__.py
  • replay_buffer.py
▼ model
  > __pycache__
  • __init__.py
  • dqn_model.py
  • play_game.py
  • train_dqn.py
```

<그림 8>

5. DQN Agent 코딩 - chessboard_for_dqn.py

- Chessboard_for_dqn.py 업데이트 요소
- GameState에 새롭게 추가된 무승부 조건: 3수 반복, 50수 규칙(폰 이동이나 기물 포획 없이 100번의 개별 이동이 진행된다면 무승부로 처리), 기물 부족 무승부
- __init__에서 추가된 요소: halfmove_clock - 50수 규칙 검사 위한 변수 / position_counts: 현재 체스판 상태를 key, 해당 상태가 게임 중 등장한 횟수를 value로 갖는 dictionary, 3수 반복 규칙에 활용

```
class GameState:
    def __init__(
        self,
        board=None,
        next_player=Player.white,
        halfmove_clock=0,
        position_counts=None,
    ):
        self.board = board if board is not None else Board()
        self.next_player = next_player
        self.halfmove_clock = halfmove_clock

        if position_counts is None:
            current_key = self.position_key()
            self.position_counts = {current_key: 1}
        else:
            self.position_counts = position_counts
```

5. DQN Agent 코딩 - chessboard_for_dqn.py

- Chessboard_for_dqn.py 업데이트 요소
- GameState에 새롭게 추가된 무승부 조건: 3수 반복, 50수 규칙(폰 이동이나 기물 포획 없이 100번의 개별 이동이 진행 되면 무승부로 처리), 기물 부족 무승부
- Position_key: 3수 반복 규칙에 활용하기 위해 게임 상태를 tuple로 저장
- is_threelfold_repetition, is_fifty_move_draw, is_insufficient_material: 무승부 규칙 담당

```
def position_key(self):
    pieces = []

    for point, item in self.board.grid.items():
        player, piece = item
        pieces.append(
            (
                point.row,
                point.col,
                player.value,
                piece.value,
            )
        )

    pieces.sort()

    return (
        self.next_player.value,
        tuple(pieces),
    )
```

```
def is_threelfold_repetition(self):
    current_key = self.position_key()
    return self.position_counts.get(current_key, 0) >= 3

def is_fifty_move_draw(self):
    return self.halfmove_clock >= FIFTY_MOVE_LIMIT

def is_insufficient_material(self):
    non_king_pieces = []

    for point, item in self.board.grid.items():
        player, piece = item

        if piece != Piece.king:
            non_king_pieces.append((point, player, piece))

    # King vs King
    if len(non_king_pieces) == 0:
        return True

    # King + Bishop vs King
    # King + Knight vs King
    if len(non_king_pieces) == 1:
        piece = non_king_pieces[0][2]
        return piece in {Piece.bishop, Piece.knight}

    # King + Bishop vs King + Bishop,
    # when both bishops are on the same-colored squares.
    if len(non_king_pieces) == 2:
        _, first_player, first_piece = non_king_pieces[0]
        _, second_player, second_piece = non_king_pieces[1]

        if (
            first_player != second_player
            and first_piece == Piece.bishop
            and second_piece == Piece.bishop
        ):
            return True
    return False
```


5. DQN Agent 코딩 - chessboard_for_dqn.py

- Chessboard_for_dqn.py 업데이트 요소
- Gamestate에 새롭게 추가된 무승부 조건: 3수 반복, 50수 규칙(폰 이동이나 기물 포획 없이 100번의 개별 이동이 진행 되면 무승부로 처리), 기물 부족 무승부
- Position_key: 3수 반복 규칙에 활용하기 위해 게임 상태를 tuple로 저장
- is_threelfold_repetition, is_fifty_move_draw, is_insufficient_material: 무승부 규칙 담당

```
def position_key(self):
    pieces = []

    for point, item in self.board.grid.items():
        player, piece = item
        pieces.append(
            (
                point.row,
                point.col,
                player.value,
                piece.value,
            )
        )

    pieces.sort()

    return (
        self.next_player.value,
        tuple(pieces),
    )
```

```
def is_threelfold_repetition(self):
    current_key = self.position_key()
    return self.position_counts.get(current_key, 0) >= 3

def is_fifty_move_draw(self):
    return self.halfmove_clock >= FIFTY_MOVE_LIMIT

def is_insufficient_material(self):
    non_king_pieces = []

    for point, item in self.board.grid.items():
        player, piece = item

        if piece != Piece.king:
            non_king_pieces.append((point, player, piece))

    # King vs King
    if len(non_king_pieces) == 0:
        return True

    # King + Bishop vs King
    # King + Knight vs King
    if len(non_king_pieces) == 1:
        piece = non_king_pieces[0][2]
        return piece in {Piece.bishop, Piece.knight}

    # King + Bishop vs King + Bishop,
    # when both bishops are on the same-colored squares.
    if len(non_king_pieces) == 2:
        _, first_player, first_piece = non_king_pieces[0]
        _, second_player, second_piece = non_king_pieces[1]

        if (
            first_player != second_player
            and first_piece == Piece.bishop
            and second_piece == Piece.bishop
        ):
            return True
    return False
```

5. DQN Agent 코딩 - chessboard_for_dqn.py

- Chessboard_for_dqn.py 업데이트 요소
- Is_draw/is_over: 무승부 조건 정리 / 게임 종료 요건에 is_draw 추가
- Apply_move, apply_move_fast, apply_move_without_check: 추가된 gamestate 요소에 맞추어 Apply_move 관련 기능 조정

```
def is_draw(self):
    return (
        self.is_stalemate()
        or self.is_threefold_repetition()
        or self.is_fifty_move_draw()
        or self.is_insufficient_material()
    )

def draw_reason(self):
    if self.is_stalemate():
        return "stalemate"

    if self.is_threefold_repetition():
        return "threefold repetition"

    if self.is_fifty_move_draw():
        return "fifty-move rule"

    if self.is_insufficient_material():
        return "insufficient material"

    return None

def is_over(self):
    return self.is_checkmate() or self.is_draw()
```

```
def apply_move(self, move):
    if move not in self.legal_moves():
        raise ValueError("illegal move")

    return self.apply_move_without_check(move)

def apply_move_fast(self, move):
    return self.apply_move_without_check(move)

def apply_move_without_check(self, move):
    moving_item = self.board.get(move.point_from)

    if moving_item is None:
        raise ValueError("there is no piece at point_from")

    moving_player, moving_piece = moving_item

    if moving_player != self.next_player:
        raise ValueError("cannot move opponent piece")

    captured_item = self.board.get(move.point_to)
    next_board = self.board.place_stone(move)
    next_player = self.next_player.other

    if moving_piece == Piece.pawn or captured_item is not None:
        next_halfmove_clock = 0
    else:
        next_halfmove_clock = self.halfmove_clock + 1

    next_position_counts = self.position_counts.copy()

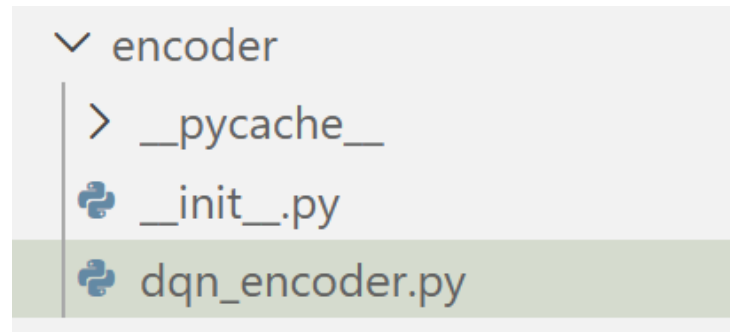
    next_state = GameState(
        board=next_board,
        next_player=next_player,
        halfmove_clock=next_halfmove_clock,
        position_counts=next_position_counts,
    )

    next_key = next_state.position_key()
    next_state.position_counts[next_key] = (
        next_state.position_counts.get(next_key, 0) + 1
    )

    return next_state
```

5. DQN Agent 코딩 - dqn_encoder.py

- Dqn_encoder: $Q(s, a)$ 신경망에 투입할 수 있도록 (S, A)를 텐서로 변환하는 파일
 - State는 (12, 8, 8) 크기의 tensor로 변환
 - 12: 우리편/상대편 pawn, knight, bishop, rook, queen, king의 위치, (8,8): chessboard 묘사
- Ex) 우리편 knight 위치가 (0,4)면 (1, 0, 4) = 1로 표기, 그 외 빈칸은 0 / 우리편, 상대편의 모든 기물 위치를 tensor로 변환
- Action은 (2, 8, 8) 크기의 tensor로 변환
 - 2: 출발 위치/도착 위치, (8,8): chessboard 묘사(참고: 이동하는 기물의 종류를 action에 포함하지 않는 이유는 state와 action 데이터를 통해 이동하는 기물의 종류에 대한 정보를 해석할 수 있기 때문)
 - 이후 encode, action tensor를 합쳐 (14, 8, 8) 크기의 tensor로 (S, A) 정보를 가공



<그림 9> dqn_encoder.py 위치

5. DQN Agent 코딩 - dqn_encoder.py

- 필요한 패키지 import, 주요 변수 저장
- Normalize_position: 흑의 경우 position이 뒤집어져 있어, 이를 보정해주는 장치
- Encode_state, encode_action: state, action 정보를 torch tensor로 변환(참고: torch tensor는 GPU 연산, 자동 미분 등을 지원해 ndarray보다 적합함)

```
import torch

from black_white.chessboard_for_dqn import Piece
from black_white.player import Player

BOARD_SIZE = 8
STATE_CHANNELS = 12
ACTION_CHANNELS = 2
COMBINED_CHANNELS = STATE_CHANNELS + ACTION_CHANNELS

PIECE_TO_INDEX = {
    Piece.pawn: 0,
    Piece.knight: 1,
    Piece.bishop: 2,
    Piece.rook: 3,
    Piece.queen: 4,
    Piece.king: 5,
}
```

```
def normalize_position(row, col, perspective):
    if perspective == Player.white:
        return row, col

    return BOARD_SIZE - 1 - row, BOARD_SIZE - 1 - col

def encode_state(game_state, perspective = None, device = None):
    if perspective == None:
        perspective = game_state.next_player

    state = torch.zeros(
        (STATE_CHANNELS, BOARD_SIZE, BOARD_SIZE),
        dtype=torch.float32,
        device=device,
    )

    for point, (player, piece) in game_state.board.grid.items():
        row, col = normalize_position(point.row, point.col, perspective)
        piece_index = PIECE_TO_INDEX[piece]

        if player == perspective:
            channel = piece_index
        else:
            channel = 6 + piece_index

        state[channel, row, col] = 1.0

    return state

def encode_action(move, perspective, device = None):
    action = torch.zeros(
        (ACTION_CHANNELS, BOARD_SIZE, BOARD_SIZE),
        dtype = torch.float32,
        device = device,
    )

    #출발 위치
    from_row, from_col = normalize_position(move.point_from.row, move.point_from.col, perspective)
    #도착 위치
    to_row, to_col = normalize_position(move.point_to.row, move.point_to.col, perspective)

    action[0, from_row, from_col] = 1.0
    action[1, to_row, to_col] = 1.0

    return action
```

5. DQN Agent 코딩 - dqn_encoder.py

- 필요한 패키지 import, 주요 변수 저장
- Encode_state_action: state, move tensor를 합쳐 (14, 8, 8) 크기 tensor를 반환
- Encode_legal_moves: 특정 상태 S(gamestate object)를 받았을 때, 가능한 move들을 전부 모아 (move 개수, 14, 8, 8) 크기의 tensor를 반환하는 함수

```
def encode_state_action(game_state, move, perspective=None, device=None):
    #state, move 텐서를 합침
    if perspective is None:
        perspective = game_state.next_player

    state = encode_state(
        game_state,
        perspective=perspective,
        device=device,
    )
    action = encode_action(
        move,
        perspective=perspective,
        device=device,
    )

    return torch.cat((state, action), dim=0)
```

```
def encode_legal_moves(
    game_state,
    legal_moves=None,
    perspective=None,
    device=None,
):
    if perspective is None:
        perspective = game_state.next_player

    if legal_moves is None:
        legal_moves = game_state.legal_moves()

    move_count = len(legal_moves)

    if move_count == 0:
        return torch.empty(
            (0, COMBINED_CHANNELS, BOARD_SIZE, BOARD_SIZE),
            dtype=torch.float32,
            device=device,
        )

    state = encode_state(
        game_state,
        perspective=perspective,
        device=device,
    )

    batch = torch.zeros(
        (
            move_count,
            COMBINED_CHANNELS,
            BOARD_SIZE,
            BOARD_SIZE,
        ),
        dtype=torch.float32,
        device=device,
    )
    batch[:, :STATE_CHANNELS] = state
    for index, move in enumerate(legal_moves):
        action = encode_action(
            move,
            perspective=perspective,
            device=device,
        )

        batch[index, STATE_CHANNELS:] = action
    return batch
```

5. DQN Agent 코딩 - replay_buffer.py

- Replay_buffer.py: DQN agent가 게임을 진행하면서 얻은 경험을 저장하고, 저장된 경험 중 일부를 무작위로 꺼내 학습에 제공하는 기능을 담당하는 파일
- Transition, Transitionbatch class: Agent가 하나의 수를 선택할 경우 만들어지는 (현재 상태 S, 선택한 행동 A, 받은 보상 R, 다음 상태 S', S'에서 가능한 행동, 게임 종료 여부)를 저장하는 Class(참고: Transitionbatch는 Transition class 여러 개를 batch 형태로 묶어 저장하는 class)

```
import random
from collections import deque
from dataclasses import dataclass
from typing import Iterable, Sequence

import torch
from torch import Tensor

ActionCoordinates = tuple[int, int, int, int]

@dataclass(frozen=True)
class Transition:
    state: Tensor
    action: Tensor
    reward: float
    next_state: Tensor
    next_legal_actions: tuple[ActionCoordinates, ...]
    done: bool

@dataclass(frozen=True)
class TransitionBatch:
    states: Tensor
    actions: Tensor
    rewards: Tensor
    next_states: Tensor
    next_legal_actions: list[tuple[ActionCoordinates, ...]]
    dones: Tensor
```

5. DQN Agent 코딩 - replay_buffer.py

- ReplayBuffer class: 학습 경험 저장, 훈련 시 mini-batch training을 위해 학습 데이터를 무작위로 추출해 mini-batch로 나누어 전달(자세히 알고 싶으시면 미니배치 경사하강법 찾아보시길 권장합니다.)

- `__init__`: capacity - ReplayBuffer object가 최대 저장할 수 있는 Transition object의 개수, `_buffer`: Transition object들을 저장하는 곳

- `__len__`, `__bool__`: `_buffer`의 길이와 비어있는지에 대한 여부를 ReplayBuffer object 자체에 대한 호출로도 가능하게 하는 보조 함수

- Push: state, next_state, next_legal_action들이 정해진 크기에 맞는지 검사(`_validate` 계열 함수)하고, Transition object를 `_buffer`에 저장

참고) `detach().cpu().clone()`: 저장하는 Tensor를 원본으로부터 분리해 cpu에 저장하도록 하고, 자동미분 등의 옵션을 제거해 메모리를 절약

```
class ReplayBuffer:
    def __init__(self, capacity: int = 50_000):
        if capacity <= 0:
            raise ValueError("capacity must be greater than 0")
        self.capacity = capacity
        self._buffer: deque[Transition] = deque(maxlen=capacity)

    def __len__(self) -> int:
        return len(self._buffer)

    def __bool__(self) -> bool:
        return bool(self._buffer)

    def push(
        self,
        state: Tensor,
        action: Tensor,
        reward: float,
        next_state: Tensor,
        next_legal_actions: Iterable[ActionCoordinates],
        done: bool,
    ) -> None:
        self._validate_state_tensor(state, "state")
        self._validate_state_tensor(next_state, "next_state")
        self._validate_action_tensor(action)

        compact_actions = tuple(
            self._validate_action_coordinates(coords)
            for coords in next_legal_actions
        )

        self._buffer.append(
            Transition(
                state=state.detach().cpu().clone(),
                action=action.detach().cpu().clone(),
                reward=float(reward),
                next_state=next_state.detach().cpu().clone(),
                next_legal_actions=compact_actions,
                done=bool(done),
            )
        )
```

5. DQN Agent 코딩 - replay_buffer.py

- ReplayBuffer class
 - sample: _buffer에 저장되어 있는 Transition object 중 batch_size개 만큼을 무작위로 추출하여 Transitionbatch object로 반환하는 함수
 - 참고) 이미 Transition object 내부에서 tensor로 선언되어있던 state, action, next_state 등은 torch.stack()으로 새로운 차원을 더해 하나의 tensor로 합침
 - 참고) 숫자, bool object였던 reward, done도 tensor로 합침
 - 참고) next_legal_action의 경우, batch 내에 포함된 대국 상황에 따라 그때 가능한 move들의 개수가 다르므로 tensor로 합치는 것은 어려움, 따라서 list 상태로 유지
 - 그 외 _validate 계열 함수들은 각 tensor의 형태를 검사하는 함수(파일 참조, 강의록에 넣지는 않았음)

```
def sample(
    self,
    batch_size: int,
    device: torch.device | str | None = None,
) -> TransitionBatch:
    if batch_size <= 0:
        raise ValueError("batch_size must be greater than 0")
    if batch_size > len(self._buffer):
        raise ValueError(
            f"batch_size ({batch_size}) is larger than "
            f"buffer size ({len(self._buffer)})"
        )

    transitions = random.sample(list(self._buffer), batch_size)

    return TransitionBatch(
        states=torch.stack([t.state for t in transitions]).to(device),
        actions=torch.stack([t.action for t in transitions]).to(device),
        rewards=torch.tensor(
            [t.reward for t in transitions],
            dtype=torch.float32,
            device=device,
        ),
        next_states=torch.stack(
            [t.next_state for t in transitions]
        ).to(device),
        next_legal_actions=[
            t.next_legal_actions for t in transitions
        ],
        dones=torch.tensor(
            [t.done for t in transitions],
            dtype=torch.bool,
            device=device,
        ),
    )
```


5. DQN Agent 코딩 - dqn_model.py

- Dqn_model.py: Q(s, a)에 쓰일 신경망 구조를 정의한 파일
 - self.feature_extractor: ((batch 개수, 14, 8, 8) -> 합성곱 신경망(nn.Conv2d) 통해 (batch, 64, 8, 8) -> ReLU 통해 비선형성 추가)*3회(다루기 편하게 self.feature_extractor로 묶어 관리) ->
 - self.q_value_head: (batch, 64*8*8) 크기로 flatten -> 선형 신경망(nn.Linear) 통해 (batch, 128) 크기로 출력 -> ReLU 적용 -> 선형 신경망 통해 (batch, 1) 출력
 - 신경망 처리는 self.feature_extractor -> self.q_value_head 순서로 데이터 처리
 - 결과적으로 batch 내 각 상황에 대해 하나의 Q값을 전달함

```
class DQNModel(nn.Module):
    def __init__(self, hidden_channels=64, hidden_features=128):
        super().__init__()

        self.feature_extractor = nn.Sequential(
            nn.Conv2d(
                in_channels=COMBINED_CHANNELS,
                out_channels=hidden_channels,
                kernel_size=3,
                padding=1,
            ),
            nn.ReLU(),
            nn.Conv2d(
                in_channels=hidden_channels,
                out_channels=hidden_channels,
                kernel_size=3,
                padding=1,
            ),
            nn.ReLU(),
            nn.Conv2d(
                in_channels=hidden_channels,
                out_channels=hidden_channels,
                kernel_size=3,
                padding=1,
            ),
            nn.ReLU(),
        )

        flattened_size = hidden_channels * BOARD_SIZE * BOARD_SIZE

        self.q_value_head = nn.Sequential(
            nn.Flatten(),
            nn.Linear(flattened_size, hidden_features),
            nn.ReLU(),
            nn.Linear(hidden_features, 1),
        )
```

5. DQN Agent 코딩 - dqn_model.py

- Dqn_model.py: $Q(s, a)$ 에 쓰일 신경망 구조를 정의한 파일
- forward: 입력값 x 를 넣어 self.feature_extractor -> self.q_value_head를 거쳐 Q값 숫자들로 구성된 batch size 길이의 1차원 tensor를 반환
- 입력값을 투입하기 전 입력값을 4차원으로 맞추어 주고, 입력값 tensor의 차원 및 크기가 적절한지 검사함

```
def forward(self, x):
    if x.dim() == 3:
        x = x.unsqueeze(0)

    if x.dim() != 4:
        raise ValueError(
            "DQNModel input must have shape "
            "(14, 8, 8) or (batch_size, 14, 8, 8).")

    expected_shape = (COMBINED_CHANNELS, BOARD_SIZE, BOARD_SIZE)
    if tuple(x.shape[1:]) != expected_shape:
        raise ValueError(
            f"Expected input shape (*, {expected_shape[0]}, "
            f"{expected_shape[1]}, {expected_shape[2]}), "
            f"but received {tuple(x.shape)}."
        )

    features = self.feature_extractor(x)
    q_values = self.q_value_head(features)

    return q_values.squeeze(-1)
```

5. DQN Agent 코딩 - dqn_agent.py

- Dqn_agent.py: DQN 알고리즘을 활용해 체스를 두고, 경험을 저장하며, 신경망을 업데이트하는 Agent를 적은 파일

- __init__: 주요 변수들 저장

- 1) player, replay_buffer, 할인율, batch_size, target용 Q 신경망 업데이트 주기 설정
- 2) Pytorch를 구동할 device 설정
- 3) Policy_network(강의록 슬라이드 13에서의 Q_online 신경망)과 target_network(Q_target 신경망) 설정

참고) load_state_dict를 통해 학습 진행 시 target_network의 초기 가중치를 policy_network과 동일하게 하고, target_network는 훈련 상태가 아니므로 .eval()을 통해 평가 모드로 전환(다만 dropout 등 일부 신경망을 건너뛰는 기능이 없기 때문에 .eval()에 큰 의미는 없음)

- 4) Optimizer에 업데이트할 파라미터와 학습률을 설정 (self.optimizer)

- 5) 손실 함수 정의(self.loss_function)

참고) SmoothL1Loss: 일반적으로 제공오차로 작동하지만, 오차가 1 이상으로 커질 경우 제곱하지 않고 선형적으로 계산하여 절댓값 오차로 작동함. 몇 개의 큰 오차값이 전체 gradient를 지나치게 크게 만드는 것을 방지함

- 6) 신경망 업데이트 횟수 저장(self.update_count)

```
class DQNAgent(Agent):
    def __init__(
        self,
        player,
        replay_buffer: ReplayBuffer,
        learning_rate: float = 1e-3,
        discount_factor: float = 0.99,
        batch_size: int = 64,
        target_update_interval: int = 1000,
        device: torch.device | str | None = None,
    ):
        super().__init__()

        self.player = player
        self.replay_buffer = replay_buffer
        self.discount_factor = discount_factor
        self.batch_size = batch_size
        self.target_update_interval = target_update_interval

        if device is None:
            device = "cuda" if torch.cuda.is_available() else "cpu"
        self.device = torch.device(device)

        self.policy_network = DQNModel().to(self.device)
        self.target_network = DQNModel().to(self.device)
        self.target_network.load_state_dict(self.policy_network.state_dict())
        self.target_network.eval()

        self.optimizer = torch.optim.Adam(
            self.policy_network.parameters(),
            lr=learning_rate,
        )
        self.loss_function = nn.SmoothL1Loss()
        self.update_count = 0
```

5. DQN Agent 코딩 - dqn_agent.py

- Dqn_agent.py: DQN 알고리즘을 활용해 체스를 두고, 경험을 저장하며, 신경망을 업데이트하는 Agent를 적은 파일
- select_move: ϵ -greedy 방식으로 수를 선택
- 1) ϵ 의 확률로 수를 무작위로 선택(학습 시 수 탐색을 위해, 대국 시에는 epsilon 값을 0으로 설정하여 조정 가능)
- 2) 1- ϵ 의 확률로 Q_policy(S, A)를 최대로 하는 A 선택
- encode_legal_moves를 통해 상태 S에서 가능한 모든 move들을 모아 (move 개수, 14, 8, 8) tensor를 생성
- Policy_network에 투입하여 q 예측값 list를 얻음
- 최대 q값을 주는 행동을 선택

참고) torch.no_grad(): 역전파를 실행하는 과정이 아니므로 gradient 관련 정보는 저장하지 않는다는 의미

참고) .eval()과 .train()의 경우 dropout 등이 없으므로 현재 코드에서는 큰 의미가 없음

```
def select_move(self, game_state, epsilon: float = 0.0):
    legal_moves = game_state.legal_moves()

    if not legal_moves:
        return None

    if random.random() < epsilon:
        return random.choice(legal_moves)

    encoded_moves = encode_legal_moves(
        game_state,
        legal_moves=legal_moves,
        perspective=self.player,
        device=self.device,
    )

    self.policy_network.eval()
    with torch.no_grad():
        q_values = self.policy_network(encoded_moves)
    self.policy_network.train()

    best_index = int(torch.argmax(q_values).item())
    return legal_moves[best_index]
```

5. DQN Agent 코딩 - dqn_agent.py

- remember: 각 수를 둘 때마다 해당 정보를 replay_buffer로 보내 Transition class로 저장할 수 있도록 하는 코드

참고) _move_to_coordinates: 흑 플레이어의 move에 포함된 출발 위치, 도착 위치의 좌표를 위아래를 뒤집어 보정해주는 기능을 함

```
def remember(
    self,
    state,
    action,
    reward: float,
    next_state,
    done: bool,
) -> None:
    state_tensor = encode_state(
        state,
        perspective=self.player,
    )
    action_tensor = encode_action(
        action,
        perspective=self.player,
    )
    next_state_tensor = encode_state(
        next_state,
        perspective=self.player,
    )

    if done:
        next_legal_actions = ()
    else:
        next_legal_actions = tuple(
            self._move_to_coordinates(move)
            for move in next_state.legal_moves()
        )

    self.replay_buffer.push(
        state=state_tensor,
        action=action_tensor,
        reward=reward,
        next_state=next_state_tensor,
        next_legal_actions=next_legal_actions,
        done=done,
    )
```

5. DQN Agent 코딩 - dqn_agent.py

- Optimize_model: Replay buffer에서 mini-batch를 꺼내 policy_network를 1번 학습시키는 함수

1) replay_buffer에서 batch_size 이상의 수가 쌓이지 않을 경우 바로 함수 종료

2) replay_buffer에서 sample 함수를 이용해 mini-batch를 추출함
(Transitionbatch class object가 추출됨)

3) 추출된 mini_batch 내에 있는 (batch_size, 12, 8, 8) 크기의 batch.states와 (batch_size, 2, 8, 8) 크기의 batch.actions를 합쳐 (batch_size, 14, 8, 8) 크기의 state_action_batch를 만듦

4) Policy_network를 통해 q 예측값들을 모든 1차원 tensor를 만듦(크기는 batch_size)

```
def optimize_model(self) -> float | None:
    if len(self.replay_buffer) < self.batch_size:
        return None

    batch = self.replay_buffer.sample(
        self.batch_size,
        device=self.device,
    )

    state_action_batch = torch.cat(
        (batch.states, batch.actions),
        dim=1,
    )
    predicted_q_values = self.policy_network(state_action_batch)

    with torch.no_grad():
        next_q_values = torch.zeros(
            self.batch_size,
            dtype=torch.float32,
            device=self.device,
        )

        for index in range(self.batch_size):
            if batch.dones[index]:
                continue

            action_coordinates = batch.next_legal_actions[index]
            if not action_coordinates:
                continue
            next_actions = self._coordinates_to_action_batch(
                action_coordinates
            )
            repeated_next_state =
            batch.next_states[index].unsqueeze(0).expand(
                len(action_coordinates), -1, -1, -1
            )
            next_inputs = torch.cat(
                (repeated_next_state, next_actions),
                dim=1,
            )
            next_q_values[index] =
            self.target_network(next_inputs).max()
            target_q_values = batch.rewards + (
                self.discount_factor
                * next_q_values
                * (~batch.dones).float()
            )
```

```
loss = self.loss_function(
    predicted_q_values,
    target_q_values,
)

self.optimizer.zero_grad()
loss.backward()
nn.utils.clip_grad_norm_(
    self.policy_network.parameters(),
    max_norm=1.0,
)
self.optimizer.step()

self.update_count += 1
if self.update_count %
self.target_update_interval == 0:
    self.update_target_network()

return float(loss.item())
```

5. DQN Agent 코딩 - dqn_agent.py

- Optimize_model: Replay buffer에서 mini-batch를 꺼내 policy_network를 1번 학습시키는 함수

5) next_q_values 저장을 위한 공간을 torch.zeros()로 마련

6) For index in range~: mini-batch 내부 (Transitionbuffer object)에 있는 각 수들을 하나씩 따짐

- 끝난 수면 다음 수를 따짐
- 다음 상태에서 가능한 action들을 불러온 후 tensor 형태로 변환 (_coordinates_to_action_batch 기능)
- (S_(t+1), A_(t+1)) 쌍을 만들기 위해 (12, 8, 8) 형태인 S_(t+1)을 (action들의 개수, 12, 8, 8) 형태로 확장하고, next_actions와 합쳐 (action들의 개수, 14, 8, 8) 형태를 만들고, next_input으로 명명
- Target_network에 next_input을 넣고 Q값을 계산 후, 최댓값을 next_q_values에 저장

```
def optimize_model(self) -> float | None:
    if len(self.replay_buffer) < self.batch_size:
        return None

    batch = self.replay_buffer.sample(
        self.batch_size,
        device=self.device,
    )

    state_action_batch = torch.cat(
        (batch.states, batch.actions),
        dim=1,
    )
    predicted_q_values = self.policy_network(state_action_batch)

    with torch.no_grad():
        next_q_values = torch.zeros(
            self.batch_size,
            dtype=torch.float32,
            device=self.device,
        )

    for index in range(self.batch_size):
        if batch.dones[index]:
            continue

        action_coordinates = batch.next_legal_actions[index]
        if not action_coordinates:
            continue
        next_actions = self._coordinates_to_action_batch(
            action_coordinates
        )
        repeated_next_state =
        batch.next_states[index].unsqueeze(0).expand(
            len(action_coordinates), -1, -1, -1
        )
        next_inputs = torch.cat(
            (repeated_next_state, next_actions),
            dim=1,
        )
        next_q_values[index] =
        self.target_network(next_inputs).max()
        target_q_values = batch.rewards + (
            self.discount_factor
            * next_q_values
            * (~batch.dones).float()
        )
```

```
loss = self.loss_function(
    predicted_q_values,
    target_q_values,
)

self.optimizer.zero_grad()
loss.backward()
nn.utils.clip_grad_norm_(
    self.policy_network.parameters(),
    max_norm=1.0,
)
self.optimizer.step()

self.update_count += 1
if self.update_count %
self.target_update_interval == 0:
    self.update_target_network()

return float(loss.item())
```

5. DQN Agent 코딩 - dqn_agent.py

- Optimize_model: Replay buffer에서 mini-batch를 꺼내 policy_network를 1번 학습시키는 함수

7) next_q_values가 완성되면 이를 batch.rewards와 결합하여 TD target tensor를 target_q_values로 저장

(참고: 만약 다음 상태가 게임이 종료된 상태라 done이 True이면, 다음 수가 없어 q값을 고려할 수 없으므로 next_q_values 값에 0을 곱함)

8) loss_function에 predicted_q_values와 target_q_values를 넣고 역전파를 실행하여 policy_network를 업데이트

9) Update 횟수가 target_update_interval의 배수가 되면 update_target_network을 통해 policy_network의 parameter를 target_network에 전달

```
def optimize_model(self) -> float | None:
    if len(self.replay_buffer) < self.batch_size:
        return None

    batch = self.replay_buffer.sample(
        self.batch_size,
        device=self.device,
    )

    state_action_batch = torch.cat(
        (batch.states, batch.actions),
        dim=1,
    )
    predicted_q_values = self.policy_network(state_action_batch)

    with torch.no_grad():
        next_q_values = torch.zeros(
            self.batch_size,
            dtype=torch.float32,
            device=self.device,
        )

        for index in range(self.batch_size):
            if batch.dones[index]:
                continue

            action_coordinates = batch.next_legal_actions[index]
            if not action_coordinates:
                continue
            next_actions = self._coordinates_to_action_batch(
                action_coordinates
            )
            repeated_next_state =
            batch.next_states[index].unsqueeze(0).expand(
                len(action_coordinates), -1, -1, -1
            )
            next_inputs = torch.cat(
                (repeated_next_state, next_actions),
                dim=1,
            )
            next_q_values[index] =
            self.target_network(next_inputs).max()
            target_q_values = batch.rewards + (
                self.discount_factor
                * next_q_values
                * (~batch.dones).float()
            )
```

```
loss = self.loss_function(
    predicted_q_values,
    target_q_values,
)

self.optimizer.zero_grad()
loss.backward()
nn.utils.clip_grad_norm_(
    self.policy_network.parameters(),
    max_norm=1.0,
)
self.optimizer.step()

self.update_count += 1
if self.update_count %
self.target_update_interval == 0:
    self.update_target_network()

return float(loss.item())
```


5. DQN Agent 코딩 - dqn_agent.py

- Save/load 함수: policy_network, target_network 등의 정보를 지정된 위치에 저장하거나 불러오는 함수
- 기타 함수: _move_to_coordinates, _normalize_position, _coordinates_to_action_batch: 위의 두 함수는 move objec의 좌표를 보정해줌, _coordinates_to_action_batch는 move objec를 (2, 8, 8) 크기의 tensor로 변환해주는 작업을 수행함 (감의록에는 포함하지 않음, 파일 참조)

```
def save(self, file_path: str | Path) -> None:
    file_path = Path(file_path)
    file_path.parent.mkdir(parents=True, exist_ok=True)

    torch.save(
        {
            "policy_network": self.policy_network.state_dict(),
            "target_network": self.target_network.state_dict(),
            "optimizer": self.optimizer.state_dict(),
            "update_count": self.update_count,
        },
        file_path,
    )

def load(self, file_path: str | Path) -> None:
    checkpoint = torch.load(
        file_path,
        map_location=self.device,
    )

    self.policy_network.load_state_dict(
        checkpoint["policy_network"]
    )
    self.target_network.load_state_dict(
        checkpoint["target_network"]
    )
    self.optimizer.load_state_dict(checkpoint["optimizer"])
    self.update_count = checkpoint.get("update_count", 0)
```

5. DQN Agent 코딩 -train_dqn.py

- Train_dqn.py: 지금까지 프로그래밍한 dqn_agent와 다른 agent의 대국을 실행하고, 관련 데이터를 생성한 후 훈련에도 관여하는 파일
 - 관련 패키지/변수들 업로드:
MATERIAL_REWARD_SCALE은 파낸 기물의 Score를 reward로 반환 시 보정하는 수(ex. Queen을 파내면 queen의 점수 900점을 1000으로 나누어 0.9의 reward로 반환)
 - Play_training_episode: train 과정에 쓰일 episode들을 생성하는 함수
- 1) GameState object를 선언하고, 양 팀 Agent들이 대국을 진행함. Agent들 중 하나는 DQNAgent

```
from agent_file.random_agent import Randombot
from agent_file.dqn_agent import DQNAgent
from black_white.chessboard_for_dqn import GameState
from black_white.player import Player
from memory.replay_buffer import ReplayBuffer
```

```
WIN_REWARD = 10.0
LOSE_REWARD = -10.0
DRAW_REWARD = 0.0
MATERIAL_REWARD_SCALE = 1000.0
```

```
def play_training_episode(
    agent: DQNAgent,
    opponent,
    epsilon: float,
    max_moves: int = 300
):
    game = GameState()
    total_reward = 0.0
    losses = []
    move_count = 0

    # DQN이 흑이면 상대 Agent가 첫 수를 둔다.
    if agent.player == Player.black and not game.is_over():
        opening_move = opponent.select_move(game)

        if opening_move is None:
            return game.winner(), total_reward, None

    game = game.apply_move_fast(opening_move)
    print(game)
    move_count += 1
```

5. DQN Agent 코딩 -train_dqn.py

- Play_training_episode

2) Game이 진행되는 동안, 먼저 agent의 수를 action으로 저장하고 착수 + 착수 이전 agent의 기물 점수 저장 -> 게임이 끝날 경우 reward 부여 -> 그렇지 않을 경우 상대 수 착수 -> 상대가 둘 수가 없는 경우 게임이 끝난 것으로 판정하고 reward 부여

```
while not game.is_over() and move_count < max_moves:
    state = game
    score_before = state.score(agent.player)

    action = agent.select_move(
        state,
        epsilon=epsilon,
    )

    if action is None:
        break

    game = state.apply_move_fast(action)
    print(game)
    move_count += 1

    # DQN의 수로 바로 게임이 끝난 경우
    if game.is_over():
        winner = game.winner()

        if winner == agent.player:
            reward = WIN_REWARD
        else:
            reward = DRAW_REWARD

        done = True

    else:
        opponent_move = opponent.select_move(game)

        if opponent_move is None:
            winner = game.winner()

            if winner == agent.player:
                reward = WIN_REWARD
            elif winner is None:
                reward = DRAW_REWARD
            else:
                reward = LOSE_REWARD

            done = True
```

5. DQN Agent 코딩 -train_dqn.py

- Play_training_episode

3) 상대 수가 결정된 후에도 게임이 끝나지 않음 -> 상대 수 착수 -> 최대 대국 한계수 넘어설 경우 무승부 처리 -> 게임이 끝나면 reward 부여, 아닐 경우 agent의 착수 이전 기물 점수와 agent/opponent의 착수 이후 agent의 기물 점수의 차이를 reward로 부여

4) state, action, reward, next_state, 게임 종료 여부를 remember로 보내 Transition class 객체에 저장

5) optimize_model을 호출하여 신경망을 1회 훈련하고 (다만 batch_size만큼의 수가 진행되지 않을 경우 훈련하지 않음), loss 값을 loss list에 저장하고 total_reward에 현재 수 reward값을 더함

참고) batch_size 크기만큼 게임이 진행되지 않고 끝나더라도, buffer 내에 수가 쌓이기 때문에 다음 게임의 수를 더하면 훈련에 필요한 데이터를 충분히 확보할 수 있음

6) 게임 종료 후 게임 승자, total_reward, average_loss를 반환

```
else:
    game = game.apply_move_fast(opponent_move)
    print(game)
    move_count += 1

    if move_count >= max_moves:
        reward = DRAW_REWARD
        done = True

    else:
        done = game.is_over()

    if done:
        winner = game.winner()

        if winner == agent.player:
            reward = WIN_REWARD
        elif winner is None:
            reward = DRAW_REWARD
        else:
            reward = LOSE_REWARD

    else:
        score_after = game.score(agent.player)
        reward = (
            score_after - score_before
        ) / MATERIAL_REWARD_SCALE

agent.remember(
    state=state,
    action=action,
    reward=reward,
    next_state=game,
    done=done,
)

loss = agent.optimize_model()
if loss is not None:
    losses.append(loss)
    total_reward += reward
if losses:
    average_loss = sum(losses) / len(losses)
else:
    average_loss = None

winner = None if move_count >= max_moves else game.winner()

return game.winner(), total_reward, average_loss
```

5. DQN Agent 코딩 -train_dqn.py

- Train 함수: play_training_episode를 통해 실행된 episode의 데이터를 모아 신경망 업데이트 과정을 실행하는 함수

1) Episode 진행 횟수(대국 횟수), buffer 크기, batch 크기, epsilon 값 조정, save_interval(dqn 신경망 저장 주기), checkpoint_path(불러올 신경망 데이터 파일 저장 위치) 관련 변수 설정(참고: episode 횟수를 시험용으로 10회만 했지만, 실제로는 약 1000회 정도 설정하는 것이 맞음)

2) 연산 실행 device, buffer, agent/opponent의 Agent 지정(참고: opponent Agent의 경우 예시는 Randombot이지만, Alphabot도 사용 가능 - 다만 초기에는 DQN agent가 실력이 좋지 않으므로 Alphabot과 대국하면 패배만 거듭할 가능성이 높아 학습이 잘 되지 않을 가능성이 있음)

3) Checkpoint_path에 저장된 신경망 정보가 있으면 load, 그렇지 않으면 새로운 model로 학습 시작

```
def train(
    episodes: int = 10,
    buffer_capacity: int = 50_000,
    batch_size: int = 64,
    epsilon_start: float = 1.0,
    epsilon_end: float = 0.05,
    epsilon_decay: float = 0.995,
    save_interval: int = 100,
    checkpoint_path = None
):
    device = torch.device(
        "cuda" if torch.cuda.is_available() else "cpu"
    )

    replay_buffer = ReplayBuffer(
        capacity=buffer_capacity,
    )

    opponent = Randombot()

    agent = DQNAgent(
        player=Player.white,
        replay_buffer=replay_buffer,
        batch_size=batch_size,
        device=device,
    )

    if checkpoint_path is not None:
        checkpoint_path = Path(checkpoint_path)

        if checkpoint_path.exists():
            agent.load(checkpoint_path)

            print(
                "Loaded checkpoint:",
                checkpoint_path,
            )
            print(
                "Restored update count:",
                agent.update_count,
            )
        else:
            print(
                "Checkpoint not found. "
                "Starting with a new model:",
                checkpoint_path,
            )
```

5. DQN Agent 코딩 -train_dqn.py

- Train 함수: play_training_episode를 통해 실행된 episode의 데이터를 모아 신경망 업데이트 과정을 실행하는 함수

4) 초기 epsilon 값을 설정 후, agent와 opponent가 번갈아가며 흑/백을 play_training_episode를 통해 플레이하도록 함. Episode가 끝난 후 승패 횟수를 갱신하고, epsilon 값을 감소시킴

```
epsilon = epsilon_start

agent_wins = 0
agent_losses = 0
draws = 0

print("Training device:", device)
print("Opponent:", opponent.__class__.__name__)

for episode in range(1, episodes + 1):
    if episode % 2 == 1:
        agent.player = Player.white
    else:
        agent.player = Player.black

    winner, total_reward, average_loss = play_training_episode(
        agent=agent,
        opponent=opponent,
        epsilon=epsilon,
    )

    if winner == agent.player:
        agent_wins += 1
    elif winner is None:
        draws += 1
    else:
        agent_losses += 1

    epsilon = max(
        epsilon_end,
        epsilon * epsilon_decay,
    )
```

5. DQN Agent 코딩 -train_dqn.py

- Train 함수: play_training_episode를 통해 실행된 episode의 데이터를 모아 신경망 업데이트 과정을 실행하는 함수

5) 특정 주기(현재 예시는 10회지만, 자유롭게 변경 가능)마다 average_loss 값을 포함한 각종 데이터 출력

6) save_interval 만큼이 진행되면 해당 시점에서의 신경망 데이터를 저장

7) 훈련이 종료된 후의 신경망 데이터를 final_path에 저장

8) __name__ == "__main__"을 통해 train()함수 호출

참고) checkpoint_path의 경우 테스트로 돌려본 데이터가 존재하여 예시에는 파일 경로를 넣었지만, 해당 위치에 신경망 데이터가 없으면 그냥 train()만 호출해도 됨

```
if episode % 10 == 0:
    if average_loss is not None:
        loss_text = f"{average_loss:.5f}"
    else:
        loss_text = "not enough data"

    print(
        f"Episode {episode:5d} | "
        f"side={agent.player.name:5s} | "
        f"reward={total_reward:7.3f} | "
        f"loss={loss_text} | "
        f"epsilon={epsilon:.3f} | "
        f"buffer={len(replay_buffer)} | "
        f"W/B/D={agent_wins}/{agent_losses}/{draws}"
    )

    if episode % save_interval == 0:
        checkpoint_path = (
            Path("checkpoints")
            / f"dqn_episode_{episode}.pt"
        )
        agent.save(checkpoint_path)

    final_path = Path("checkpoints") / "dqn_final.pt"
    agent.save(final_path)

if __name__ == "__main__":
    train(checkpoint_path = "~/checkpoints/dqn_final.pt")
```

6. train_dqn.py 실행 결과

- 예시로 episode 횟수를 10회 설정하고 학습을 실행해본 결과, 각 episode의 진행 상황과 학습 관련 데이터가 출력되었고, 훈련을 통해 얻은 신경망의 가중치가 별도의 폴더에 저장되는 것을 확인할 수 있었다.

```
  0 1 2 3 4 5 6 7
7  . . . . . q . .
6  . . . . . . . K
5  . . . . . . . .
4  . . . . . p . .
3  . . . . . p . .
2  p . . . . P p k
1  . . . . . . . .
0  . . . . . . . .

  0 1 2 3 4 5 6 7
7  . . . . . q . .
6  . . . . . . . .
5  . . . . . K . .
4  . . . . . p . .
3  . . . . . p . .
2  p . . . . P p k
1  . . . . . . . .
0  . . . . . . . .

  0 1 2 3 4 5 6 7
Episode   10 | side=black | reward=  1.200 | loss=0.01028 | epsilon=0.951 | buffer=1229 | W/B/D=1/2/7
```

✓ checkpoints
|  dqn_final.pt

<그림 9>

6. 앞으로의 개선 과제

- Q-learning 외에도 Actor-critic algorithm과, 이를 발전시킨 alphazero 버전의 알고리즘 등 다양한 강화학습 알고리즘이 존재함. 더 강한 체스 Agent를 만들기 위해서는 이러한 알고리즘 활용 가능
- 개인 컴퓨터의 경우 몇천 회가 넘는 episode를 실행하는 것은 어려울 것임. Ssh 접속을 통해 클라우드 서버에 연결해서 훈련을 실행하거나, 동아리 컴퓨터를 활용하는 방안이 필요함(동아리 컴퓨터를 활용하고 싶을 경우, 저 또는 예과 부장인 한재준(24학번) 학우에게 문의하시면 될 것 같습니다.)
- 더 효율적으로 episode를 훈련하기 위해, cpu 코어별로 작업을 분산하고, 신경망 처리는 cuda 등을 통해 gpu에 할당하는 방법이 존재함
- 각 조에서 제공된 파일을 기반으로 자유롭게 개선해 나가 주시면 될 것 같습니다..!

감사합니다!